

ALGORITMOS E PENSAMENTO COMPUTACIONAL



Prof. Dr. Nelson Missaglia





ALGORITMOS E PENSAMENTO COMPUTACIONAL

- ✓ Introdução à Programação em C
- ✓ Elaborado com o auxílio de Gemini Google
- ✓ Slide Master ofertado pelo Prof. Marco

Reflexão



Algoritmo pra quê?

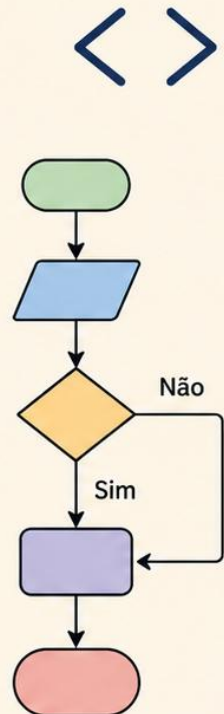
```
#include <stdio.h>

int main(void) {
    printf("Olá, C!\n");
    return 0;
}
```

PRIMEIRO, RESOLVA O PROBLEMA.

DEPOIS, ESCREVA O CÓDIGO.

John Johnson



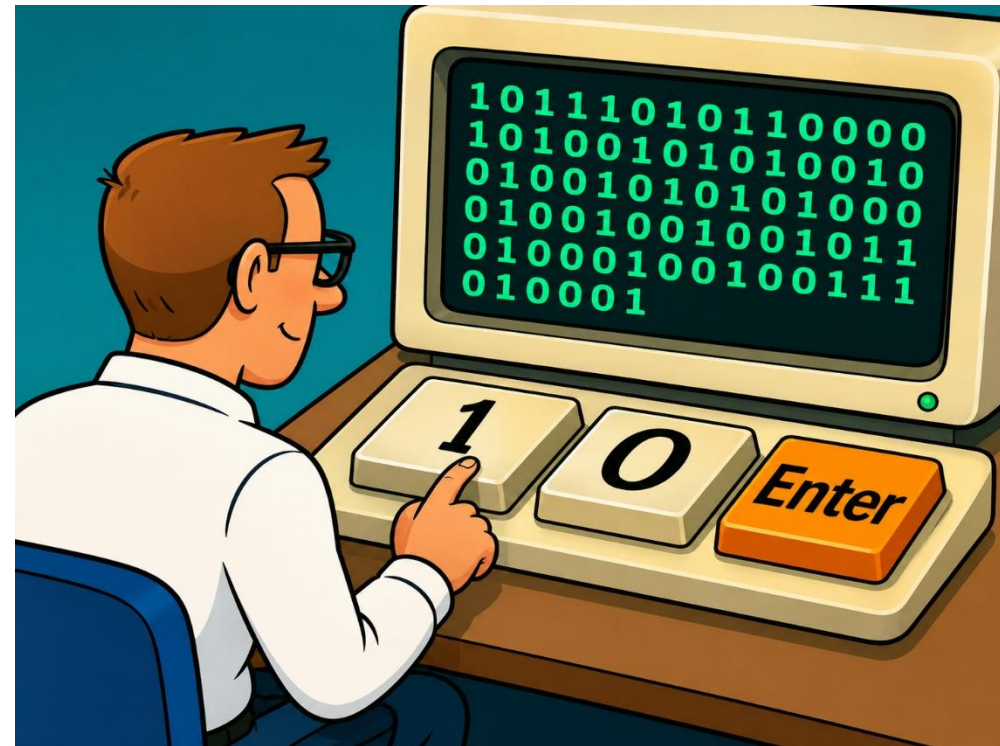
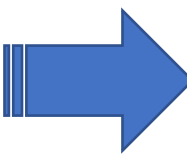
O que é um Programa de Computador?

São instruções escritas em uma linguagem que permite a comunicação entre o programador e o computador (0's e 1's) → Linguagem de programação

```
000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000001000001000
0000000101011000111100011000000011
11111000110101001111001111001111
00011010000110001111010101111000
11100000111111000011111101010111
11100000111111000011111101010111
```

Código de máquina

Um arquivo contendo instruções em linguagem de máquina é chamado de **executável**.



Linguagens de programação

Ferramentas para comunicação entre pessoas e computadores



Uma linguagem natural ou idioma (português, inglês, espanhol, etc.) estabelece um vocabulário ou dicionário válido e um grupo de regras sintáticas e semânticas para que as pessoas se expressem adequadamente nessa linguagem e se comuniquem.



De forma semelhante, uma linguagem de programação estabelece um conjunto de regras sintáticas e semânticas, para escrever códigos válidos, a fim de que os computadores possam executar uma determinada função.

 **Em resumo:** Linguagens naturais conectam pessoas. Linguagens de programação conectar pessoas e computadores.

Exemplos de Linguagens de Programação



JavaScript



Ruby



C#



Java



C++



C



Python



PHP



Cada linguagem possui suas próprias **palavras** (vocabulário) e **regras de uso**, o que permite que o código seja compreendido pelo computador.

Por que usar uma linguagem de programação?

As linguagens de programação constituem o mecanismo fundamental através do qual os algoritmos abstratos são convertidos em instruções operacionais interpretáveis pelos sistemas computacionais:



A linguagem de programação é o meio pelo qual **transformamos algoritmos em instruções compreensíveis** pelo computador, permitindo a execução real da solução;



Programar é **dar vida ao algoritmo**, transformando algo escrito em linguagem natural, fluxograma ou pseudocódigo, em instruções que o computador compreende;



É transformar **raciocínio lógico em ação automatizada**.

Algoritmo
(abstrato)



Ideia e lógica da
solução
independente de
linguagem.

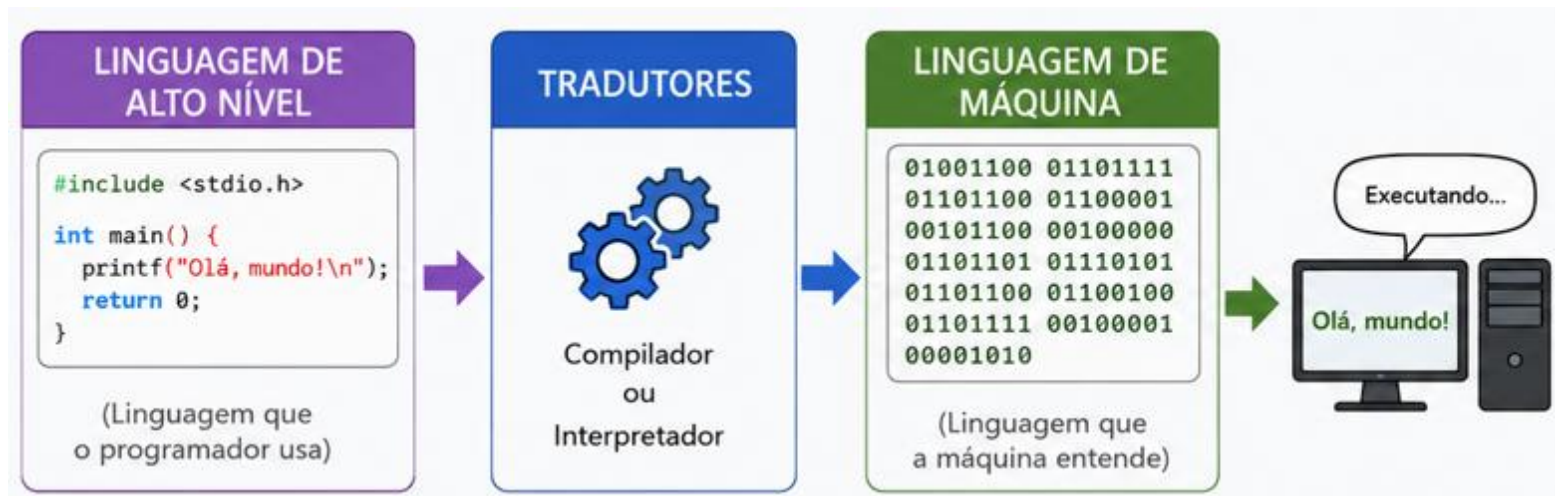
Linguagem de
Programação



Converte o
algoritmo em
instruções
compreensíveis
pelo computador.

Como a máquina entende os códigos?

Para que o computador “entenda” um programa é necessário um meio de tradução entre a linguagem de alto nível utilizada no programa e a linguagem de máquina.



Para essa tarefa temos basicamente dois métodos:

- **Compilador:** traduz o programa escrito em uma linguagem de programação para um programa executável.
- **Interpretador:** traduz e executa o programa linha por linha



Linguagens de programação

As linguagens de programação classificam-se principalmente pelo **nível de abstração**, que é a proximidade com o hardware ou com o ser humano e pelo **paradigma de programação**, que é a forma como estruturam as instruções e os dados.

Cada tipo atende a necessidades específicas do desenvolvimento de software.



Retrato em aquarela de [Ada Lovelace](#), possivelmente feito por [Alfred Edward Chalon](#) em torno de 1840.

Linguagens de programação



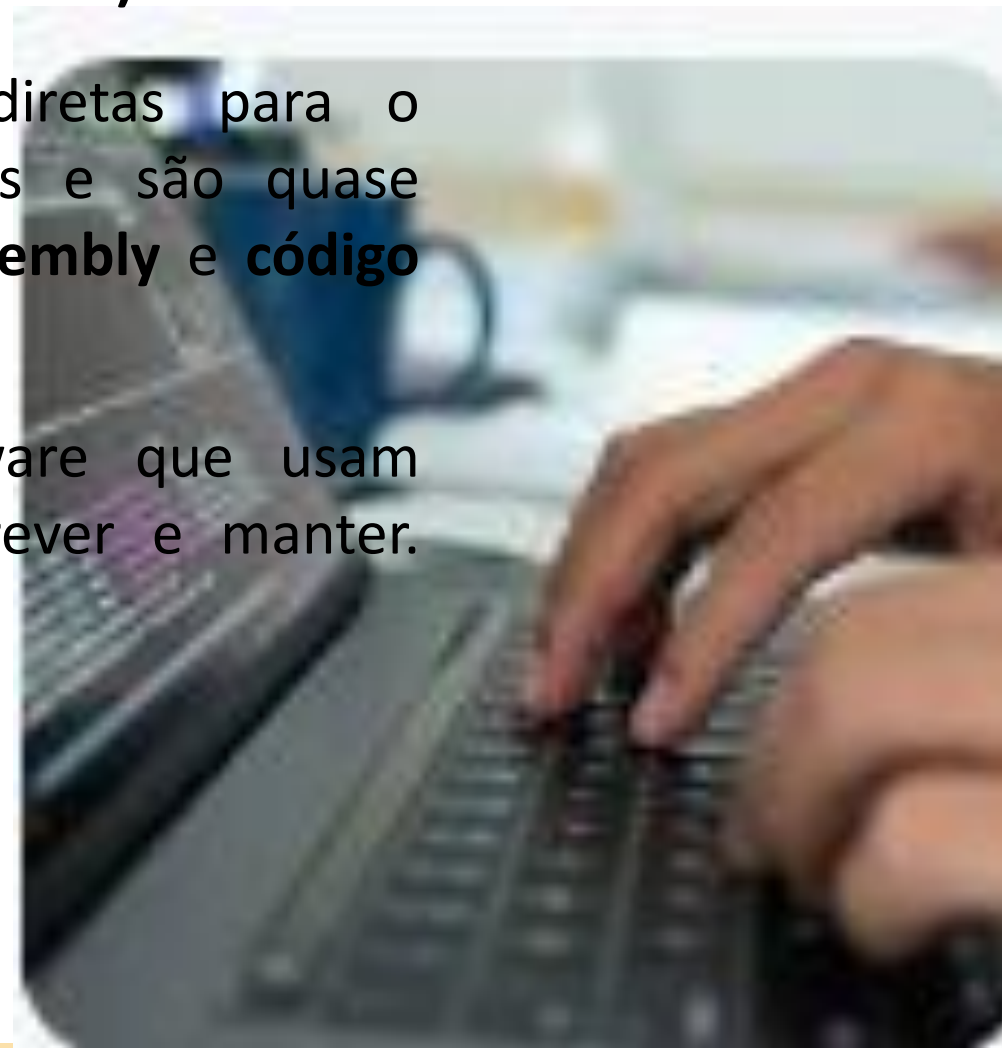
Ada Lovelace (1815–1852) foi uma matemática e escritora inglesa, amplamente reconhecida como a **primeira programadora da história**. Filha do famoso poeta Lord Byron, ela combinou sua herança literária com a lógica matemática para conceber o primeiro algoritmo do mundo para a "Máquina Analítica" de Charles Babbage. (Brasil Escola)

Linguagens de programação

1. Níveis de Abstração (Proximidade com a Máquina)

- **Baixo Nível:** Linguagens com instruções diretas para o processador. Não possuem termos em inglês e são quase incompreensíveis para humanos. Exemplo: **Assembly** e **código binário**.

- **Alto Nível:** Linguagens distantes do hardware que usam comandos em inglês. São fáceis de ler, escrever e manter. Exemplos: [Python](#), [Java](#) e JavaScript.



Linguagens de programação

2. Paradigmas de Programação (Modelos de Desenvolvimento)

Os paradigmas definem a abordagem lógica usada pelo programador para estruturar o código.

- **Imperativo:** Foca em *como* a tarefa deve ser feita, listando comandos passo a passo para o computador alterar seu estado.

Exemplo: **C**.

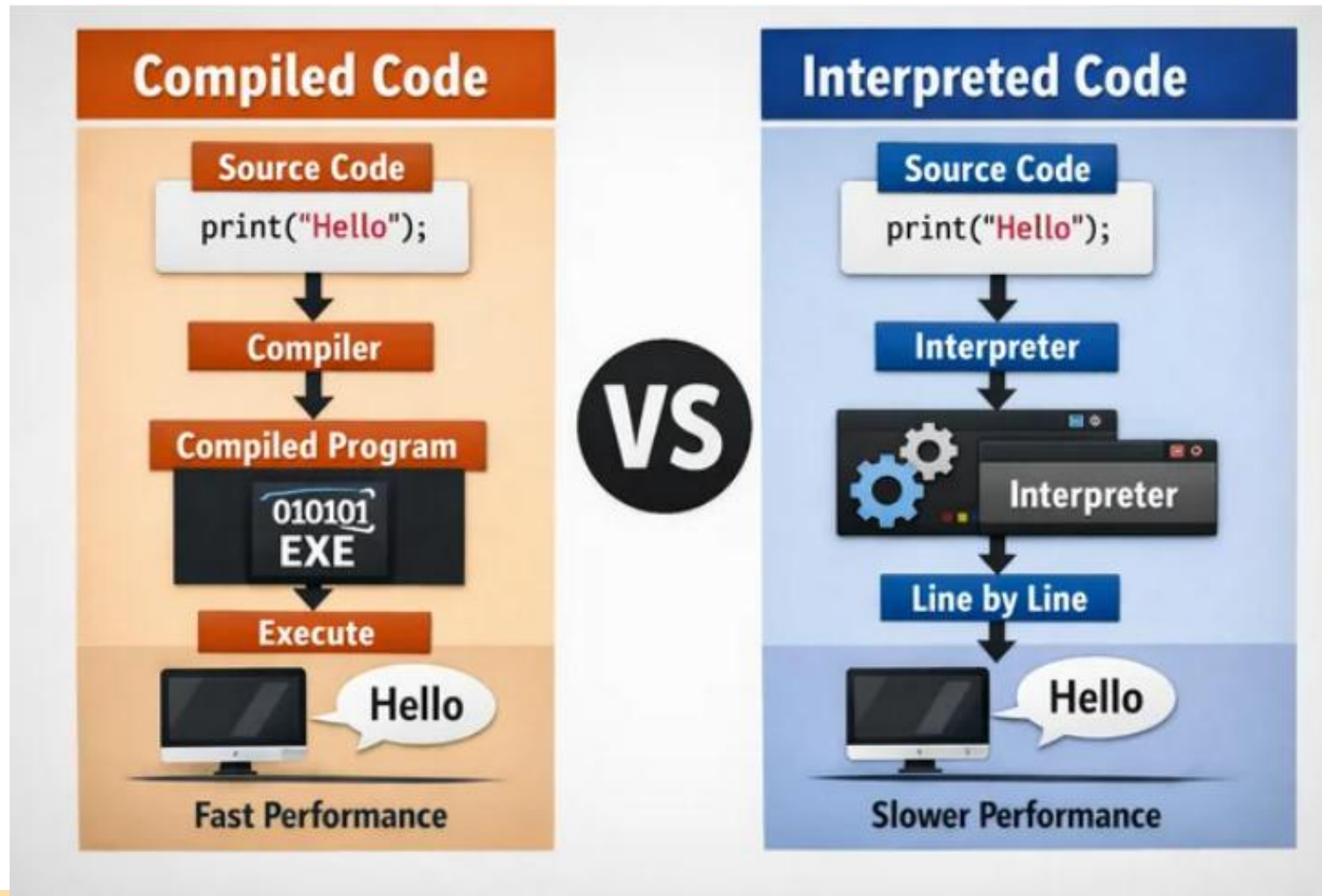
- **Orientado a Objetos (POO):** Organiza o código em "objetos" da vida real, agrupando dados e funções em uma mesma estrutura. Exemplos: [Java](#) e C#.

Linguagens de programação

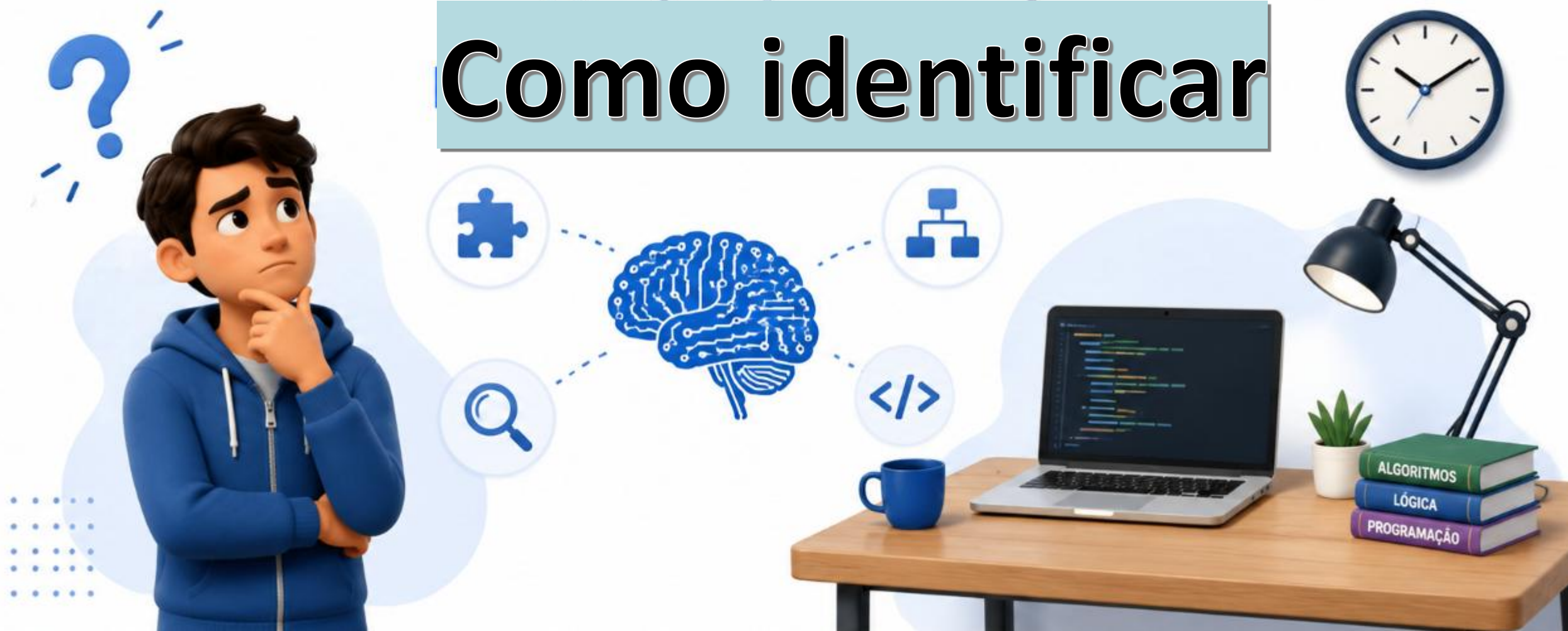
- **Funcional:** Trata a computação como a avaliação de funções matemáticas isoladas, evitando dados mutáveis. Exemplos: **Haskell** e **Elixir**.
- **Lógico (Declarativo):** Foca no *o que* deve ser resolvido em vez de como resolver. O programa usa fatos e regras para deduzir respostas. Exemplo: **Prolog**.

Linguagens de programação

- Linguagens Compiladas X Interpretadas.



Como identificar



Linguagens de programação

Linguagens de programação **compiladas** traduzem todo o código-fonte em um arquivo executável de máquina antes de rodar, tornando a execução mais rápida.

Linguagens **interpretadas** traduzem o código linha por linha, em tempo real, utilizando um programa chamado interpretador, o que facilita testes e a portabilidade.

Linguagens de programação

Linguagens Compiladas

- **Vantagens:** O programa roda de forma muito mais rápida, pois já está perfeitamente traduzido para o hardware. Ele é distribuído como um arquivo executável, ocultando o código-fonte original.
- **Desvantagens:** Qualquer alteração no código exige uma nova compilação completa. Além disso, um programa compilado para Windows não rodará no Linux ou macOS, exigindo compilações separadas para cada sistema.
- **Exemplos:** C, C++, Rust e Go.

<https://devskin.com/post/linguagem-compilada-ou-interpretada>

<https://medium.com/@erickamil/004-java-cap%C3%ADtulo-0-aula-4-compila%C3%A7%C3%A3o-interpret%C3%A7%C3%A3o-a731ace52316>

<https://cicerocq.wordpress.com/wp-content/uploads/2018/08/introdcomputadores.pdf>

Linguagens de programação

Linguagens Interpretadas

- **Vantagens:** Alta portabilidade. O mesmo arquivo de código pode rodar em qualquer sistema operacional que possua o interpretador compatível. Também facilita a correção de erros e testes rápidos.
- **Desvantagens:** O tempo de execução é geralmente mais lento, já que o interpretador precisa traduzir o código enquanto o programa está rodando.
- **Exemplos:** Python, JavaScript, PHP e Ruby.

<https://devskin.com/post/linguagem-compilada-ou-interpretada>

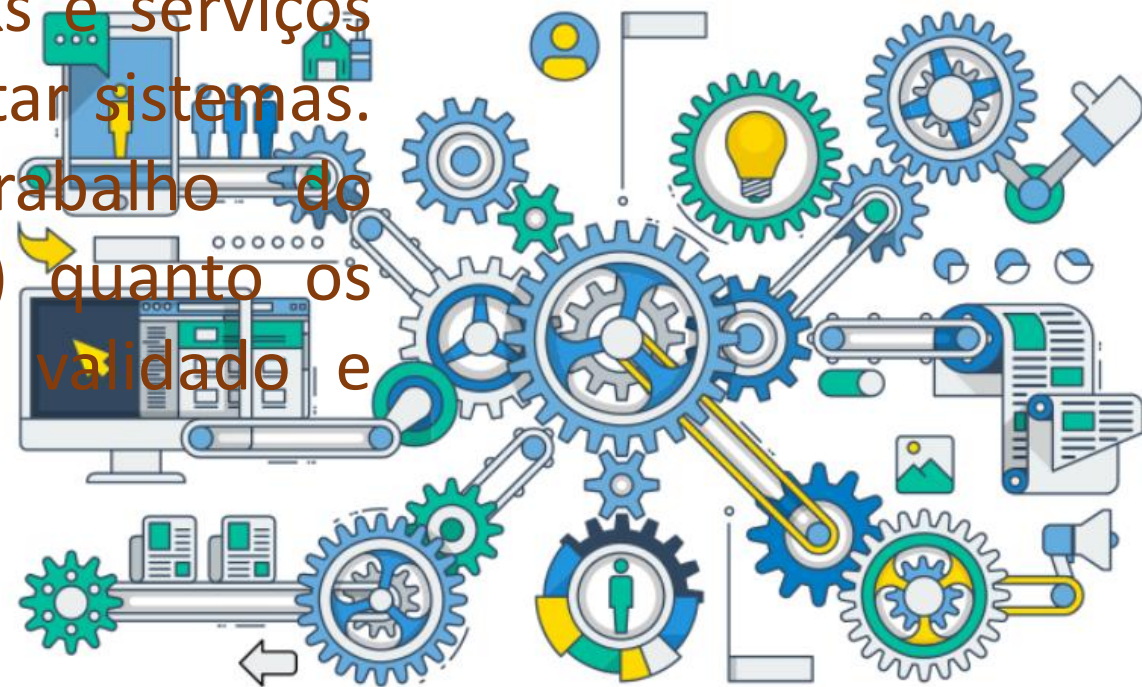
<https://medium.com/@erickamil/004-java-cap%C3%ADtulo-0-aula-4-compila%C3%A7%C3%A3o-interpretada%C3%A7%C3%A3o-a731ace52316>

<https://cicerocq.wordpress.com/wp-content/uploads/2018/08/introdcomputadores.pdf>

Linguagens de programação

Ambiente de desenvolvimento de software

Um ambiente de desenvolvimento de software é a infraestrutura de ferramentas, frameworks e serviços usada para programar, testar e implementar sistemas. Ele engloba tanto o espaço de trabalho do programador na própria máquina (local) quanto os servidores remotos onde o software é validado e entregue ao usuário final.



https://ead4.fgv.br/producao/DI/FUNDACAO_BRADESCO/scorm/scorm_bt29_2017_05_26/pag/2_2_1.html

Ambientes para desenvolvimento de programas (IDE)

São ferramentas que facilitam que fornecem recursos para edição, compilação e execução de programas e outros recursos importantes.



NetBeans e Eclipse

Principais IDE para desenvolvimento na plataforma Java. Ambas podem ser utilizadas para outras linguagens, como o C.



Editor de código avançado



Depurador integrado



Suporte a plugins



Gerenciamento de projetos



Microsoft Visual Studio Code

Editor de código-fonte gratuito, leve e extremamente popular desenvolvido pela Microsoft, compatível com Windows, macOS e Linux, além de possuir uma versão executável diretamente no navegador web.



Leve e rápido



IntelliSense e autocompletar



Integração com Git e GitHub



Extensões ilimitadas



Execução no navegador



Dev++ e Code::Blocks

IDE clássico, gratuito e de código aberto, amplamente utilizado por iniciantes e estudantes para programar nas linguagens C e C++.



Simple e fácil de usar



Compilação com diversos compiladores



Execução com um clique



Gerenciamento de projetos



Leve e eficiente

Linguagens de programação

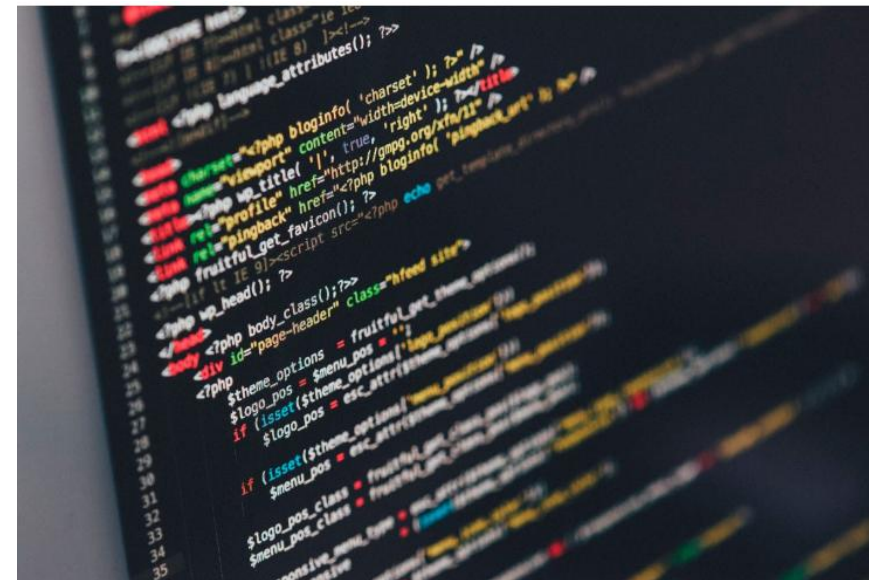
Editores de código

Editores de código são softwares desenvolvidos especificamente para escrever e editar código-fonte. Diferente de editores simples, eles possuem recursos como destaque de sintaxe, autocompletar e integração com controle de versão.

Exemplo: Dev C++

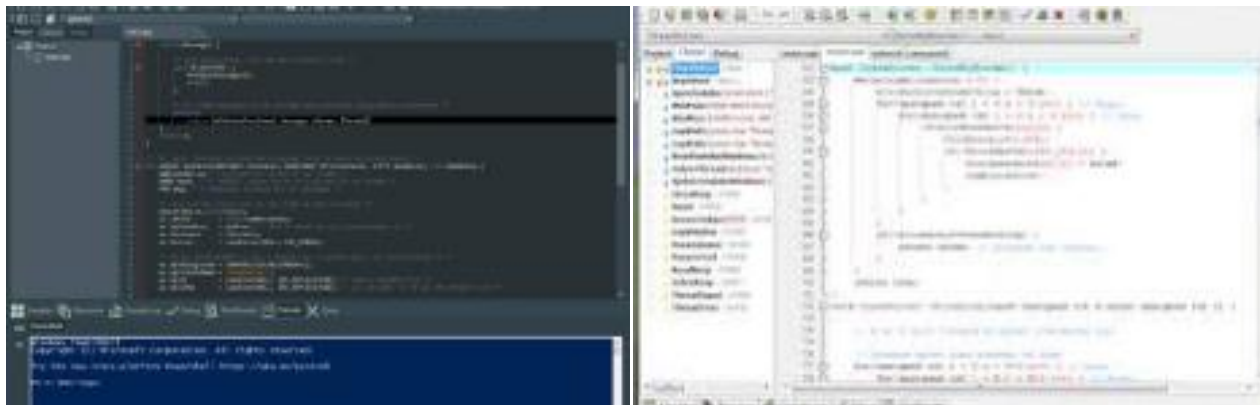


<https://www.uva.br/postagens/principais-editores-codigo-mercado/>



Linguagens de programação

O **Dev-C++** é um Ambiente de Desenvolvimento Integrado (IDE) **gratuito, leve e de código aberto** utilizado para programar nas linguagens **C e C++** no sistema operativo Windows. Ele destaca-se pelo seu baixo consumo de memória, inicialização rápida e por já incluir um compilador (geralmente o GCC/MinGW), o que permite escrever e testar código imediatamente após a instalação.



Linguagens de programação

Sintaxe e semântica de linguagens de programação

Sintaxe e semântica são os dois pilares fundamentais que estruturam as linguagens de programação, definindo, respectivamente, a forma correta de escrever o código e o significado computacional dessas instruções.



Linguagem C



Nesta disciplina utilizaremos a **linguagem C** para implementação de **alguns algoritmos**.



A linguagem de programação C foi desenvolvida no início dos anos 1970, motivada pela necessidade de uma linguagem **mais poderosa e flexível** para o desenvolvimento do sistema operacional **UNIX**.



C é uma linguagem **estruturada**, o que facilita o aprendizado dos principais conceitos da programação, como variáveis, funções, estruturas de controle e operadores.



Permite um **acesso mais direto** aos recursos do computador, como memória e dispositivos, sendo ideal para quem deseja entender o que realmente acontece "por trás do código".



C continua sendo amplamente utilizada em **sistemas operacionais, aplicativos de alto desempenho, drivers, compiladores, embarcados** e muito mais!

Características da linguagem C



Linguagem rápida e eficiente



Portável entre diferentes sistemas



Estruturada e modular



Base para muitas outras linguagens

exemplo.c

```
1  #include <stdio.h>
2
3  int main() {
4      int idade;
5      printf("Digite sua idade: ");
6      scanf("%d", &idade);
7
8      printf("Sua idade é %d anos.\n", idade);
9
10     return 0;
11 }
```

Linguagem C: aplicações no mundo real



A linguagem C é amplamente utilizada em:



Sistemas operacionais (como o Linux);



Sistemas embarcados;



Automação industrial e de processos;



Compiladores e Interpretadores.



Base para outras linguagens:

Aprender C ajuda a formar uma base sólida para transitar com facilidade por outras linguagens modernas, como **C++, Java, Python** e até mesmo linguagens específicas de automação.



Estrutura básica de um programa em C

Inclui arquivos de cabeçalho necessários ao programa

Permite usar comandos de entrada e saída, como escrever mensagens na tela (**stdio.h**)

```
#include <stdio.h>
```

Função responsável pelo início da execução do programa (**main**)

```
int main(){  
    printf("Olá mundo\n");  
    return 0;  
}
```

Bloco de instruções



A maioria dos programas iniciais em C terá uma estrutura semelhante a essa.

Linguagens de programação

O que é Sintaxe?

- **Regras de escrita:** Define como os símbolos, palavras e caracteres devem ser combinados para formar um código válido.
- **Forma e gramática:** É como a "ortografia" da linguagem. Por exemplo, exigir o uso de ponto e vírgula (;) no final de uma linha ou parênteses em uma função.
- **Detecção de erros:** Erros de sintaxe são fáceis de encontrar; o computador avisa imediatamente durante a compilação ou leitura se você escreveu algo fora da regra.

Linguagens de programação

O que é Semântica?

- **Significado computacional:** Atribui um sentido prático e o que o comando realmente faz quando executado pelo computador.
- **Lógica e ação:** Determina a função da instrução, como somar dois números, exibir um texto na tela ou guardar um valor na memória.
- **Detecção de erros:** Erros semânticos ocorrem quando o código está escrito de forma gramaticalmente certa, mas a lógica faz o computador realizar uma tarefa errada ou impossível (como dividir um número por zero).

Tipos de Dados e Variáveis em Linguagem C

Em **Linguagem C**, variáveis são espaços reservados na memória para armazenar dados, e cada uma deve possuir um **tipo de dado fixo e declarado previamente**. Como C é uma linguagem fortemente e estaticamente tipada, o tipo define o tamanho que a variável ocupará na memória e como os bits armazenados serão interpretados.

tipo	Descrição	Tamanho
bool	Valores Aceitos true ou false	1 byte ou 8 bits
char	Intervalos de -128 a 127	1 byte ou 8 bits
signed char	Intervalos de -128 a 127	1 byte ou 8 bits
unsigned char	Intervalos de 0 a 255	1 byte ou 8 bits
int	Intervalos de -2.147.483.648 a 2.147.483.647	4 bytes ou 32 bits
short int	Intervalos de -32.768 a 32.767	2 bytes ou 16 bits
signed int	Intervalos de -2.147.483.648 a 2.147.483.647	4 bytes ou 32 bits
unsigned int	Intervalos de 0 a 4.294.967.295	4 bytes ou 32 bits
unsigned short int	Intervalos de 0 a 4.294.967.295	2 bytes ou 16 bits
float	Intervalos de 1,2e-38 até 3,4e+38.	4 bytes ou 32 bits
double	Intervalos de 2,2e-308 a 1,8e+308	8 bytes ou 64 bits
long double		
long	Intervalos de -2.147.483.648 a 2.147.483.647	4 bytes ou 32 bits
signed long	Intervalos de -2.147.483.648 a 2.147.483.647	4 bytes ou 32 bits
unsigned long	Intervalos de 0 a 4.294.967.295	4 bytes ou 32 bits
long long		
wchar_t	Intervalos de -32.768 a 32.767	4 bytes ou 32 bits
signed wchar_t	Intervalos de -32.768 a 32.767	4 bytes ou 32 bits
unsigned wchar_t	Intervalo de 0 a 65.535	4 bytes ou 32 bits

Tipos de Dados e Variáveis em Linguagem C

Tipos de Dados Primitivos

A linguagem C possui quatro tipos de dados básicos:

Palavra-chave	Tipo de Dado	Descrição	Tamanho Comum
<code>char</code>	Caractere	Armazena um único caractere ou símbolo.	1 byte
<code>int</code>	Inteiro	Armazena números inteiros positivos ou negativos.	4 bytes
<code>float</code>	Real (Precisão Simples)	Números com casas decimais (ponto flutuante).	4 bytes
<code>double</code>	Real (Precisão Dupla)	Números decimais com o dobro de precisão do float.	8 bytes
<code>void</code>	Vazio	Indica a ausência de um tipo ou de valor.	0 bytes

Tipos de Dados e Variáveis em Linguagem C

Modificadores de Tipo

Os modificadores alteram o tamanho ou a forma como o sinal dos tipos primitivos (especialmente o `int` e o `char`) é interpretado na memória:  IBM +2

- `signed` : Permite números positivos e negativos (padrão do sistema).
- `unsigned` : Aceita apenas números positivos, dobrando a capacidade positiva.
- `short` : Reduz o espaço de memória reservado para a variável.
- `long` : Aumenta a capacidade de armazenamento da variável.  Instituto de Computação +4

Tipos de Dados e Variáveis em Linguagem C

Regras para Criação de Variáveis

Para nomear variáveis em C, é obrigatório seguir estas regras de sintaxe:

- **Apenas letras, números e sublinhados (`_`):** Não use espaços ou símbolos especiais.
- **Início obrigatório:** Deve começar sempre com uma letra ou sublinhado, nunca com números.
- **Case Sensitive:** A linguagem diferencia maiúsculas de minúsculas (`idade` é diferente de `Idade`).
- **Palavras reservadas:** Símbolos e instruções nativas da linguagem (como `int`, `float`, `return`) não podem ser nomes de variáveis.

Estrutura básica de um programa em C



No início do programa, normalmente colocamos **bibliotecas, constantes e outras declarações** que poderão ser usadas ao longo do código.



Todo programa em C deve conter uma função **main()**, responsável pelo início da execução do programa, onde colocamos os comandos que queremos executar.



Usamos **int main(void)** porque a função devolve um número inteiro ao sistema operacional. O comando **return 0** indica que o programa terminou corretamente.



As chaves **{}** delimitam o bloco de instruções que será executado pela função.



A maioria das instruções em C termina com **;** (ponto e vírgula).



Exemplos C - exemplo_inicial.c

```
1 #include <stdio.h>
2 #define PI 3.14159
3
4 int main() {
5     float raio, area;
6     printf("Digite o raio: ");
7     scanf("%f", &raio);
8     area = PI * raio * raio;
9     printf("Área = %.2f\n", area);
10    return 0;
11 }
```



Declarações

Bibliotecas, constantes e outras declarações globais.



Função principal

Ponto de entrada do programa.



Bloco de instruções

Comandos executados dentro da função main().



Finalização

return 0 indica término correto do programa.



Fim do bloco

As chaves delimitam o bloco da função.



Em C, clareza e organização do código são essenciais! Cada parte do programa tem um papel importante.

Comando de saída em C



A função **printf()** permite que os campos de exibição sejam formatados. As formatações mais usadas são o preenchimento com zeros à esquerda, para inteiros, e a especificação do número de casas decimais, para reais.

Exemplo 1.6. Formatação de campos com *printf()*.

```
int a = 678;
float b = 12.3456;
...
printf("\n|%5d|", a);
printf("\n|%06d|", a);
printf("\n|%7.3d|", b);
printf("\n|%7.2d|", b);
...
```



Observações

- A largura inclui os dígitos, o sinal (se houver) e o ponto decimal.
- Para inteiros, a especificação ".x" é ignorada.

Especificadores utilizados

Especificador	Significado	Aplica-se a	Exemplo
%5d	Campo com largura mínima de 5 posições. Preenchimento com espaços à esquerda.	Inteiros	678
%06d	Campo com largura mínima de 6 posições. Preenchimento com zeros à esquerda.	Inteiros	000678
%7.3d	Número real com largura mínima de 7 posições e 3 casas decimais.	Reais	12.346
%7.2d	Número real com largura mínima de 7 posições e 2 casas decimais.	Reais	12.35

Vamos trabalhar um pouquinho?!!!



Tipos de Dados e Variáveis em Linguagem C

Exemplo Prático de Declaração e Atribuição

O código abaixo exemplifica como declarar, inicializar e exibir variáveis em um programa estruturado:

```
c
#include <stdio.h>

int main() {
    // Declaração e inicialização de variáveis
    int idade = 26;
    float altura = 1.75;
    char inicial = 'A';
    unsigned int id_usuario = 45500;
    // Exibindo os valores na tela
    printf("Idade: %d anos\n", idade);
    printf("Altura: %.2f metros\n", altura);
    printf("Inicial do nome: %c\n", inicial);
    printf("ID do Usuário: %u\n", id_usuario);

    return 0;
}
```

Tipos de Dados e Variáveis em Linguagem C

Em **Linguagem C**, os tipos primitivos são a base para qualquer programa. Ao contrário de linguagens modernas, C **não possui um tipo booleano nativo clássico** em sua especificação original, mas resolve isso de forma simples.

1. Inteiros (`int`)

Armazenam números inteiros positivos ou negativos.

- **Sintaxe:** `int contador = 10;`

- **Tamanho padrão:** 4 bytes (na maioria das arquiteturas modernas).

- **Modificadores**

- `unsigned int`: Apenas números positivos (dobra o limite positivo).
- `short int`: Ocupa menos espaço (geralmente 2 bytes). `long int`: Ocupa mais espaço (geralmente 8 bytes)

Tipos de Dados e Variáveis em Linguagem C

2. Reais / Ponto Flutuante (`float` e `double`)

Armazenam números com casas decimais.

- **`float`**: Precisão simples.

- **Sintaxe**: `float nota = 8.5f;` (o 'f' indica que é um float).
- **Tamanho**: 4 bytes.

- **`double`**: Precisão dupla (mais comum e preciso).

- **Sintaxe**: `double pi = 3.1415926535;`
- **Tamanho**: 8 bytes.

Tipos de Dados e Variáveis em Linguagem C

3. Caracteres (`char`)

Armazena um único caractere textualmente, mas o C o processa internamente como um **número inteiro** baseado na tabela ASCII.

- **Sintaxe:** `char letra = 'A';` (sempre com aspas simples).
- **Tamanho:** 1 byte.
- **Curiosidade:** Você pode fazer operações matemáticas com `char` (ex: `'A' + 1` resulta em `'B'`).

Tipos de Dados e Variáveis em Linguagem C

4. Booleanos (bool)

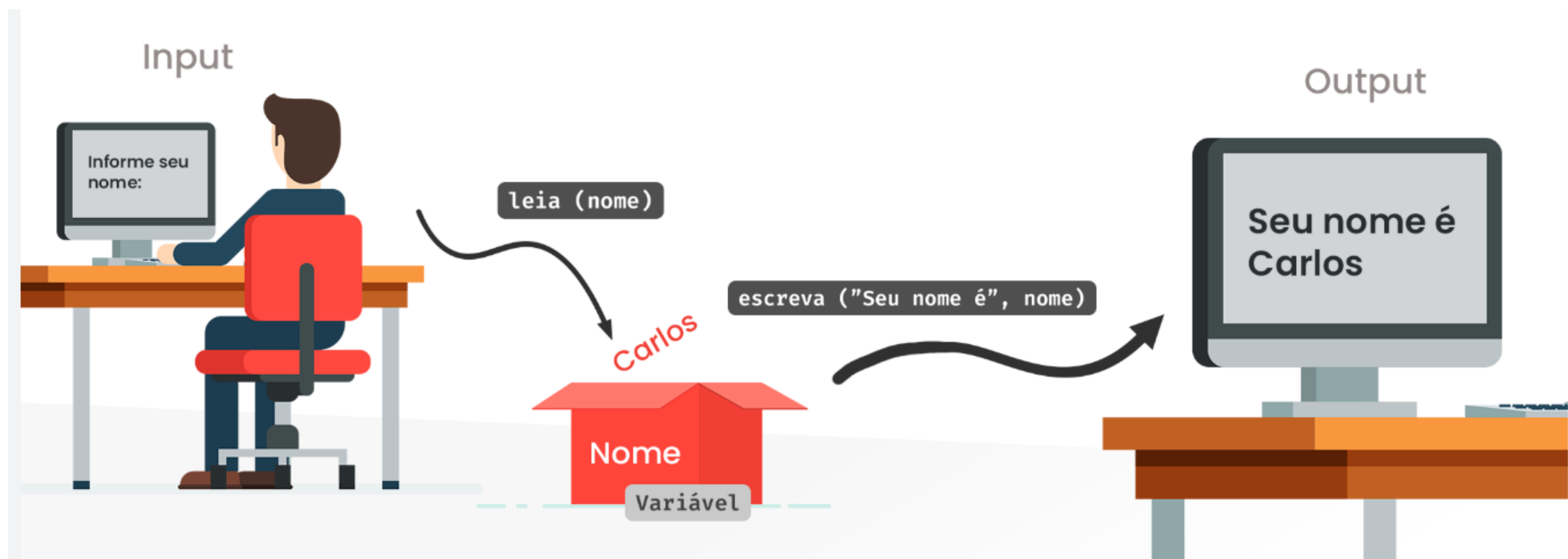
Originalmente, C usa inteiros para lógica: **0 é falso** e **qualquer outro número é verdadeiro**.

A partir do padrão C99, foi introduzido o suporte oficial:

- **Biblioteca:** É necessário incluir `#include <stdbool.h>`.
- **Sintaxe:** `bool ativo = true;` ou `bool ativo = false;`.
- **Tamanho:** 1 byte.

Tipos de Dados e Variáveis em Linguagem C

Em C, o processo de entrada e saída (E/S) de dados baseia-se na biblioteca padrão `stdio.h` (Standard Input/Output). A linguagem trata os dados como fluxos de bytes (*streams*), associando o teclado à entrada padrão (*stdin*) e o ecrã à saída padrão (*stdout*).



A seguir, apresenta-se o funcionamento dos principais mecanismos de interação com o utilizador.

Tipos de Dados e Variáveis em Linguagem C

Entrada de Dados (Receber Informações)

A função primária para capturar dados do teclado é a `scanf()`. Ao contrário da `printf()`, a `scanf()` precisa do **endereço de memória** da variável onde o dado será guardado, utilizando o operador `&` (E comercial).

- `scanf("%d", &idade);` -> Lê um número inteiro e guarda-o na variável `idade`.

- `scanf("%f", &preco);` -> Lê um número real e guarda-o na variável `preco`.

 **Exceção Importante:** Ao ler cadeias de caracteres (*strings*) com `%s`, não se usa o operador `&`, pois o nome do *array* já representa o seu endereço de memória (ex: `scanf("%s", nome);`).

Tipos de Dados e Variáveis em Linguagem C

Saída de Dados (Mostrar Informações)

A função mais utilizada para exibir dados no ecrã é a `printf()`. Ela requer o uso de **especificadores de formato** para indicar o tipo de dado que será impresso.

- `printf("Texto a ser exibido");` -> Exibe texto simples.
- `printf("Idade: %d", idade);` -> O `%d` é substituído por uma variável inteira.

Tipos de Dados e Variáveis em Linguagem C

Principais Especificadores de Formato

- `%d` ou `%i` -> Números inteiros (com sinal).
- `%f` -> Números reais (ponto flutuante `float` ou `double`).
- `%c` -> Um único caráter.
- `%s` -> Cadeias de caracteres (*strings*).

Sequências de Escape Comuns

- `\n` -> Move o cursor para a linha seguinte.
- `\t` -> Insere uma tabulação horizontal.

Tipos de Dados e Variáveis em Linguagem C

Resumo de Formato para Impressão (`printf`)

Para exibir cada tipo, use o especificador correto:

- `%d` ou `%i`: Inteiros (`int`)
- `%f`: Reais (`float`)
- `%lf`: Reais de dupla precisão (`double`)
- `%c`: Caracteres (`char`)

Tipos de Dados e Variáveis em Linguagem C

Outras Funções de E/S Comuns

Para além de `printf` e `scanf`, existem funções específicas para caracteres e texto:

- `putchar(character)` ; -> Escreve um único carácter no ecrã.
- `getchar()` ; -> Lê um único carácter do teclado.
- `puts(string)` ; -> Exibe um texto seguido automaticamente por uma nova linha (`\n`).
- `fgets(buffer, tamanho, stdin)` ; -> Forma segura de ler texto com espaços, evitando o transbordo do *buffer* (*buffer overflow*).

Vamos trabalhar mais um pouquinho?!!!



Tipos de Dados e Variáveis em Linguagem C

Exemplo Prático Completo: O código seguinte ilustra a declaração, leitura e exibição de diferentes tipos de dados:

```
#include <stdio.h>

int main() {
    int idade;
    float altura;
    char inicial;

    // Solicitação e Leitura de dados
    printf("Digite a inicial do seu nome: ");
    scanf(" %c", &inicial); // O espaço antes de %c limpa o buffer do teclado

    printf("Digite a sua idade: ");
    scanf("%d", &idade);

    printf("Digite a sua altura (ex: 1.75): ");
    scanf("%f", &altura);

    // Apresentação dos dados guardados
    printf("\n--- Dados Registrados ---\n");
    printf("Inicial: %c\n", inicial);
    printf("Idade: %d anos\n", idade);
    printf("Altura: %.2f metros\n", altura); // .2f limita a duas casas decimais

    return 0;
}
```


Exercícios de aplicação



Uma empresa de paisagismo está elaborando o projeto de um jardim retangular. Para instalar uma cerca ao redor do jardim, é necessário conhecer o seu perímetro, e para calcular a quantidade de grama que será utilizada, é preciso determinar a área do terreno.

2- Desenvolva um programa em C que leia o comprimento e a largura de um jardim retangular (em metros); calcule o perímetro do jardim utilizando a fórmula: **perímetro = 2 * (comprimento + largura)** e a área do jardim utilizando a fórmula: **área = comprimento * largura**. Ao final exiba os resultados com duas casas decimais.

Entrada:

```
> Qual o comprimento do jardim (m)?  
> 12.50  
> Qual a largura do jardim (m)?  
> 8.00
```

Entrada:

```
> Perímetro do jardim: 41.00 m  
> Área do jardim: 100.00 m²
```

Até a próxima pessoal!

49



nelson.missaglia@cruzeirodosul.edu.br



Universidade

Cruzeiro do Sul

CRÉDITOS

Material desta aula, elaborado por:

Prof. Nelson Missaglia

Material base elaborado por:

- Prof. Nelson Missaglia
- Prof. Alcides
- Prof. Lédon
- Prof. Ana Paula
- Profa. Cristiane
- Gemini Google

CONTATOS



nelson.missaglia@cruzeirodosul.edu.br

Última atualização: 2026/2